

MediLogic

Tarea 1 · Avances del Proyecto 1

Sistema experto preliminar para orientación diagnóstica mediante
Python y Prolog

Diego Ahtohil Noj Armira
Carné 201220159 · Sección B
21 de agosto de 2026

Repositorio: github.com/DiegoArmira/-IA1_2S2026_201220159

1. Resumen del avance

Se implementó una versión preliminar funcional de MediLogic. El avance integra una base de conocimiento en Prolog, un backend escrito en Python, cinco consultas controladas desde Python hacia SWI-Prolog y un prototipo del módulo de pacientes. El sistema tiene fines exclusivamente académicos y no sustituye la atención médica profesional.

Arquitectura mínima

Paciente	Frontend	Backend Python	Motor Prolog
Selecciona síntoma	Envía solicitud	Construye consulta	Infiere resultados

Componentes entregados

Componente	Descripción
prolog/medilogic.pl	Hechos de enfermedades, síntomas, medicamentos y contraindicaciones; reglas de diagnóstico, seguridad y urgencia.
backend/prolog_service.py	Puente de ejecución que invoca SWI-Prolog y limita las consultas disponibles.
backend/app.py	Servidor HTTP y endpoint /api/query implementados con Python estándar.
frontend/	Formulario preliminar para seleccionar síntoma y severidad.
GitHub Actions	Instala SWI-Prolog, valida Python y ejecuta automáticamente las cinco consultas.

2. Base de conocimiento Prolog

La base declarativa modela relaciones entre enfermedades, síntomas, medicamentos y condiciones que contraindican su uso. La regla posible_enfermedad/2 infiere diagnósticos candidatos a partir de síntomas; medicamento_seguro/4 filtra tratamientos según alergias y condiciones crónicas.

```
% MediLogic - base de conocimiento preliminar para la Entrega No. 1.
% Uso academico: orientacion diagnostica; no sustituye atencion medica.

enfermedad(gripe).
enfermedad(resfriado).
enfermedad(migrana).

sintoma(gripe, fiebre).
sintoma(gripe, tos).
sintoma(gripe, dolor_muscular).
sintoma(resfriado, congestion_nasal).
sintoma(resfriado, estornudos).
sintoma(resfriado, tos).
sintoma(migrana, dolor_cabeza).
sintoma(migrana, nausea).
sintoma(migrana, sensibilidad_luz).

medicamento(gripe, paracetamol).
medicamento(resfriado, loratadina).
medicamento(migrana, ibuprofeno).
medicamento(migrana, paracetamol).

contraindicado(ibuprofeno, gastritis).
contraindicado(ibuprofeno, alergia_aines).
contraindicado(paracetamol, enfermedad_hepatica).

% Una enfermedad es posible cuando comparte al menos un sintoma reportado.
posible_enfermedad(Sintoma, Enfermedad) :-
    sintoma(Enfermedad, Sintoma).

% Un medicamento es seguro si trata la enfermedad, no es alergeno y no esta
% contraindicado para una condicion cronica reportada.
medicamento_seguro(Enfermedad, Alergia, Condicion, Medicamento) :-
    medicamento(Enfermedad, Medicamento),
    Medicamento \= Alergia,
    \+ contraindicado(Medicamento, Condicion).

% Urgencia preliminar basada en la severidad declarada.
urgencia(severo, consulta_medica_inmediata).
```

La severidad se representa con pesos 1, 2 y 3 para leve, moderado y severo, respectivamente. Este avance deja preparada la estructura para el cálculo de afinidad ponderada solicitado en el proyecto final.

3. Backend e integración Python–Prolog

El backend no interpreta directamente la lógica clínica. Su responsabilidad es recibir parámetros, seleccionar una consulta permitida y ejecutar el motor SWI-Prolog sobre el archivo `medilogic.pl`. Esta separación conserva la inferencia en Prolog, tal como solicita el enunciado.

```
class PrologError(RuntimeError):
    """Error controlado al consultar el motor logico."""

def query(goal: str) -> list[str]:
    """Ejecuta una meta Prolog y devuelve cada solucion como texto."""
    executable = shutil.which("swipl")
    if not executable:
        raise PrologError("SWI-Prolog no esta instalado o no esta en PATH.")

    command = [
        executable,
        "-q",
        "-s",
        str(KNOWLEDGE_BASE),
        "-g",
        f"forall({goal}), (write_canonical(resultado), write('='), writeln({goal})),halt.",
    ]
    completed = subprocess.run(command, capture_output=True, text=True, timeout=10)
    if completed.returncode != 0:
        raise PrologError(completed.stderr.strip() or "La consulta Prolog fallo.")
    return [line.strip() for line in completed.stdout.splitlines() if line.strip()]

def run_named_query(name: str, params: dict[str, str]) -> dict[str, object]:
    """Construye una de cinco consultas permitidas evitando metas arbitrarias."""
    goals = {
        "enfermedades": "enfermedad(E)",
        "sintomas": f"sintoma({params.get('enfermedad', 'gripe')}, S)",
        "diagnostico": f"posible_enfermedad({params.get('sintoma', 'tos')}, E)",
        "medicamentos": f"medicamento({params.get('enfermedad', 'gripe')}, M)",
        "seguridad": (
            "medicamento_seguro("
            f"{params.get('enfermedad', 'migraña')}, "
            f"{params.get('alergia', 'ninguna')}, "
            f"{params.get('condicion', 'ninguna')}, M)"
        ),
    }
    if name not in goals:
        raise PrologError("Consulta no admitida.")
```

El uso de un catálogo cerrado de consultas evita recibir metas Prolog arbitrarias desde el cliente. Además, los errores de instalación o ejecución se convierten en mensajes controlados mediante `PrologError`.

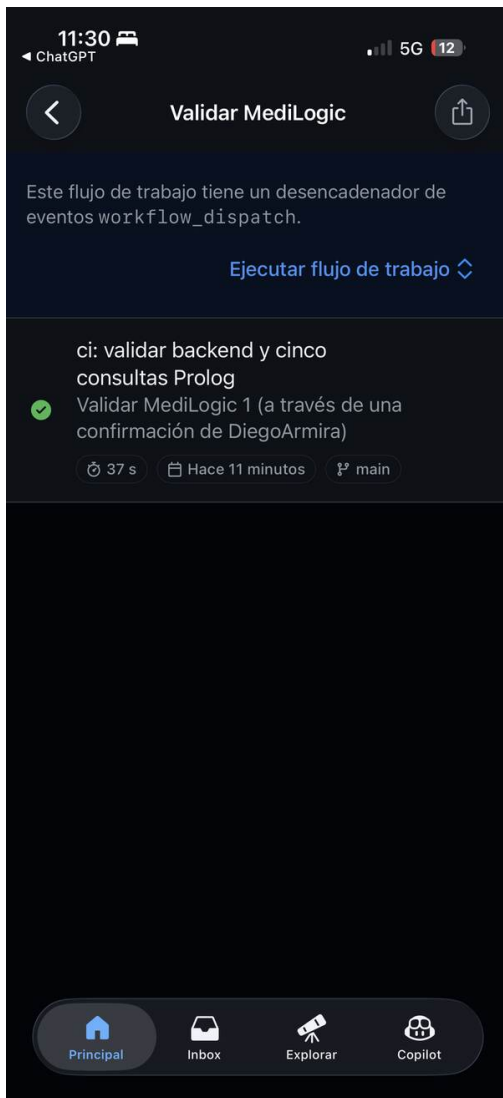
4. Cinco consultas e inferencias

Consulta	Meta ejecutada	Resultado interpretado
1. Enfermedades	enfermedad(E)	Enumera la base inicial: gripe, resfriado y migraña.
2. Síntomas	sintoma(gripe,S)	Recupera fiebre, tos y dolor muscular asociados con gripe.
3. Diagnóstico	posible_enfermedad(tos,E)	Infiere gripe y resfriado porque ambas enfermedades contienen el síntoma tos.
4. Medicamentos	medicamento(migrana,M)	Obtiene ibuprofeno y paracetamol como tratamientos registrados para migraña.
5. Seguridad	medicamento_seguro(migrana,ninguna,g astritis,M)	Descarta ibuprofeno por su contraindicación con gastritis y conserva paracetamol.

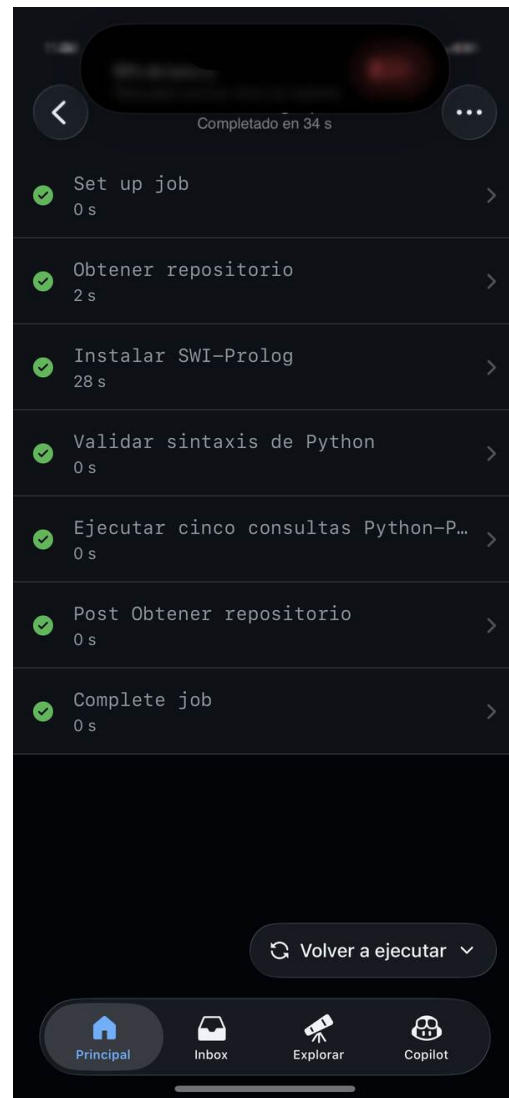
La quinta consulta demuestra inferencia y no solo recuperación de hechos: Prolog evalúa la regla de seguridad, comprueba la contraindicación y excluye el medicamento que representa conflicto.

5. Evidencia de ejecución

GitHub Actions ejecutó el backend sobre Ubuntu, instaló SWI-Prolog, validó la sintaxis de Python y completó las cinco consultas sin errores.



Flujo Validar MediLogic completado correctamente.



Pasos de instalación, validación y consultas con resultado exitoso.

6. Evidencia de resultados

```
1 ▶ Run python prolog_service.py
4 {
5   "consulta": "enfermedades",
6   "meta": "enfermedad(E)",
7   "resultados": [
8     "resultado=enfermedad(gripe)",
9     "resultado=enfermedad(resfriado)",
10    "resultado=enfermedad(migrana)"
11  ]
12 }
13 {
14   "consulta": "sintomas",
15   "meta": "sintoma(gripe, S)",
16   "resultados": [
17     "resultado=sintoma(gripe, fiebre)",
18     "resultado=sintoma(gripe, tos)",
19     "resultado=sintoma(gripe, dolor_muscular)"
20   ]
21 }
22 {
23   "consulta": "diagnostico",
24   "meta": "posible_enfermedad(tos, E)",
25   "resultados": [
26     "resultado=posible_enfermedad(tos, gripe)",
27     "resultado=posible_enfermedad(tos, resfria
28   ]
29 }
30 {
31   "consulta": "medicamentos"
```

Consultas 1 a 3: enfermedades, síntomas y diagnóstico.

```
16   "resultados": [
17     "resultado=sintoma(gripe, fiebre)",
18     "resultado=sintoma(gripe, tos)",
19     "resultado=sintoma(gripe, dolor_muscular)"
20   ]
21 }
22 {
23   "consulta": "diagnostico",
24   "meta": "posible_enfermedad(tos, E)",
25   "resultados": [
26     "resultado=posible_enfermedad(tos, gripe)",
27     "resultado=posible_enfermedad(tos, resfria
28   ]
29 }
30 {
31   "consulta": "medicamentos",
32   "meta": "medicamento(migrana, M)",
33   "resultados": [
34     "resultado=medicamento(migrana, ibuprofeno)",
35     "resultado=medicamento(migrana, paracetamo
36   ]
37 }
38 {
39   "consulta": "seguridad",
40   "meta": "medicamento_seguro(migrana, ninguna)",
41   "resultados": [
42     "resultado=medicamento_seguro(migrana, ni
43   ]
44 }
```

Consultas 4 y 5: medicamentos y validación de seguridad.

7. Control de versiones y conclusión

El repositorio presenta incrementos funcionales y trazables. Los commits principales separan la base lógica, integración, backend, interfaz, estilos, conexión del cliente, documentación y validación automática.

Commit	Propósito
feat: agregar base de conocimiento inicial	Hechos y reglas Prolog.
feat: integrar consultas Python con Prolog	Puente hacia SWI-Prolog.
feat: crear servidor backend mínimo	API y servidor HTTP.
feat: agregar interfaz del módulo de pacientes	Formulario inicial.
style: diseñar interfaz preliminar	Presentación visual.
feat: conectar interfaz con el backend	Solicitud de diagnóstico.
docs: completar documentación	Instalación y estructura.
ci: validar backend y cinco consultas Prolog	Ejecución reproducible en GitHub Actions.

Conclusión

El avance satisface el núcleo técnico de la Tarea 1: existe un backend ejecutable, integración efectiva entre Python y Prolog, cinco consultas distintas con resultados coherentes, evidencia visual de ejecución y un historial superior al mínimo de cinco commits. La siguiente iteración deberá ampliar los catálogos clínicos, el cálculo completo de afinidad, la interfaz y la documentación del proyecto final.

Repositorio público: https://github.com/DiegoArmira/-IA1_2S2026_201220159